A3 – Entrega 2: Documentação Técnica do Scanner de Quadro com IA

04/11/2026

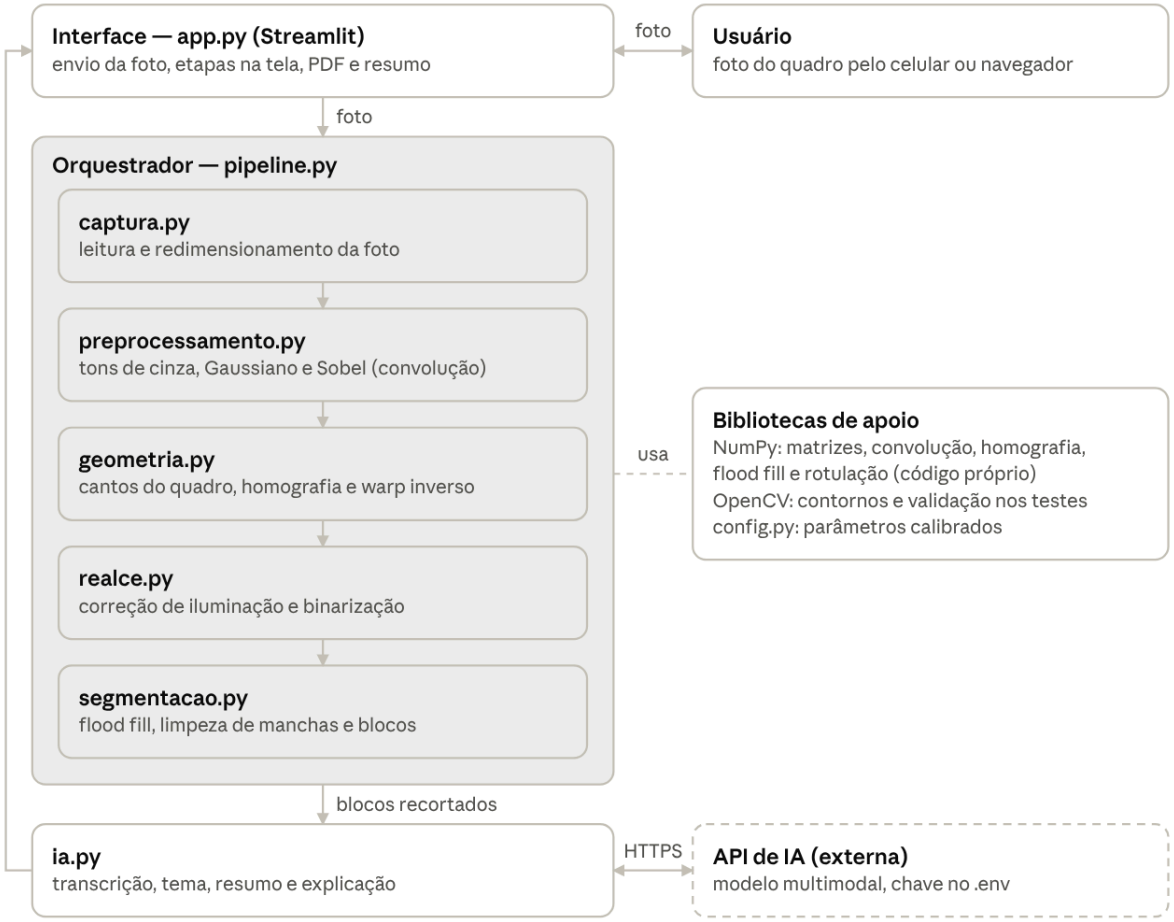
Este documento especifica como o Scanner de Quadro com IA será construído: arquitetura em seis módulos, pipeline de oito etapas, pseudocódigo dos algoritmos centrais e as bibliotecas adotadas. A fundamentação matemática de cada etapa está no relatório da Entrega 1.

Campo	**Informação**
Instituição	UNIFACS
Disciplina	Estruturas Matemáticas e Computação Gráfica
Professores	Alberlan Lopes, Inácio Borges, Norberto Maciel
Semestre	2026.2
Entrega	2 — Fase de Engenharia e Documentação Técnica
Integrantes	Artur Jansen Almeida de Souza; Davi Jansen Almeida de Souza; Luiz

1. Arquitetura do sistema

O sistema é uma aplicação Python com interface web: a interface recebe a foto, o orquestrador a passa por cinco módulos de visão computacional e, só no fim, o módulo de IA gera o resumo.

A foto passa por cinco módulos de visão antes de chegar à IA



A linha à esquerda é o retorno: a imagem tratada (em PDF) e o resumo voltam para a interface. A API de IA é o único componente externo; sem ela, o sistema ainda entrega a imagem tratada.

Módulo	Responsabilidade	Entrada	Saída
app.py	Interface, envio da foto, exibição das etapas e download	Foto do usuário	PDF + resumo na tela
pipeline.py	Chama os módulos na ordem e trata erros	Foto	Imagem limpa, blocos e texto
captura.py	Leitura e redimensionamento	Arquivo ou câmera	Matriz RGB
preprocessamento.py	Tons de cinza, suavização e bordas	Matriz RGB	Cinza e mapa de bordas
geometria.py	Cantos, homografia e retificação	Bordas + imagem	Quadro retificado
realce.py	Iluminação e binarização	Quadro retificado	Imagem corrigida e binária
segmentacao.py	Flood fill, limpeza e blocos	Imagem binária	Imagem limpa e recortes
ia.py	Prompts e chamada à API	Recortes	Tema, resumo e explicação

2. Pipeline de captura e tratamento de imagens

A foto passa por oito etapas em sequência; cada uma recebe uma matriz NumPy e devolve outra, o que permite testar e exibir cada etapa isoladamente.

Etapa	Módulo	Entrada	Operação	Saída	Parâmetros iniciais
0. Captura	captura	Arquivo JPG/PNG ou câmera	Leitura e redimensionamento (maior lado = 1600 px)	Matriz RGB M×N×3, uint8	lado_max = 1600
1. Tons de cinza	preprocessamento	RGB M×N×3	f = 0,299R + 0,587G + 0,114B	Cinza M×N	—
2. Suavização	preprocessamento	Cinza M×N	Convolução com Gaussiano 3×3	Cinza M×N	máscara 3×3
3. Bordas	preprocessamento	Cinza suavizada	Sobel Gx, Gy e magnitude + limiar	Mapa de bordas binário M×N	limiar_borda = 60
4. Cantos do quadro	geometria	Mapa de bordas	Maior contorno aproximado por 4 vértices; ordenação dos cantos	4 pontos (x, y)	área mínima = 20% da foto
5. Retificação	geometria	RGB + 4 cantos	Monta A h = b, resolve H, warp inverso com interpolação bilinear	Quadro retificado W×H	W/H pela média dos lados
6a. Iluminação	realce	Cinza do quadro retificado	Fundo b = média k×k; $\hat{f}$ = 255·min(1, f/b)	Cinza corrigida W×H	k = 41
6b. Binarização	realce	Cinza corrigida	Limiar adaptativo (média m×m − C)	Binária B W×H (1 = escrita)	m = 25, C = 10
7a. Limpeza	segmentacao	B	Rotulação por flood fill; apaga componentes com área < a_min	B’ limpa	a_min = 15 px
7b. Blocos	segmentacao	B’	Dilatação + rotulação + retângulos envolventes, ordenados de cima para baixo	Lista de recortes	dilatação 15×15
8. IA	ia	Recortes + imagem limpa	Transcrição, tema, resumo e explicação	Texto estruturado (JSON)	modelo multimodal
9. Exportação	interface	Imagem limpa + texto	Gera PDF e mostra na tela	PDF + resumo	—

Os parâmetros são valores iniciais e serão calibrados com um conjunto de 20 a 30 fotos reais de quadros da faculdade (branco e negro, ângulos e luzes diferentes). O modo de depuração salva a saída de cada etapa, o que também serve para a demonstração ao vivo.

3. Pseudocódigo dos algoritmos centrais

Os nove algoritmos abaixo cobrem o caminho completo da foto ao resumo; os de convolução, homografia, flood fill e rotulação serão implementados pelo grupo em NumPy, e o OpenCV servirá para conferir os resultados.

3.1 Pipeline principal

```
função PROCESSAR_FOTO(arquivo):
    rgb    ← CARREGAR(arquivo, lado_max = 1600)
    cinza  ← TONS_DE_CINZA(rgb)
    suave  ← CONVOLUCAO(cinza, GAUSS_3x3)
    bordas ← SOBEL(suave) > limiar_borda
    cantos ← ACHAR_CANTOS(bordas)
    se cantos = NULO:
        cantos ← PEDIR_CANTOS_AO_USUARIO(rgb)      // modo manual
    H      ← HOMOGRAFIA(cantos, retangulo_destino(cantos))
    quadro ← WARP_INVERSO(cinza, H, W, Hdest)
    claro  ← CORRIGIR_ILUMINACAO(quadro, k = 41)
    B      ← BINARIZAR_ADAPTATIVO(claro, m = 25, C = 10)
    B      ← LIMPAR_MANCHAS(B, a_min = 15)
    blocos ← SEPARAR_BLOCOS(B, claro)
    texto  ← RESUMIR_COM_IA(blocos)
    retorne (claro, B, texto)
```

3.2 Convolução 2D

```
função CONVOLUCAO(f, w):                    // f: M×N, w: (2a+1)×(2b+1)
    g ← matriz M×N de zeros
    fp ← PREENCHER_BORDA(f, a, b)           // replica as bordas
    para x de 0 até M−1:
        para y de 0 até N−1:
            soma ← 0
            para s de −a até a:
                para t de −b até b:
                    soma ← soma + w[s+a][t+b] · fp[x−s+a][y−t+b]
            g[x][y] ← soma
    retorne g
```

Na implementação, os dois laços internos viram uma soma de fatias deslocadas da matriz (vetorização NumPy), mantendo exatamente a mesma fórmula. Para Sobel, usa-se a versão separável: uma convolução com $[1, 2, 1]^T$ e outra com $[-1, 0, 1]$.

3.3 Detecção e ordenação dos cantos

```
função ACHAR_CANTOS(bordas):
    contornos ← CONTORNOS_EXTERNOS(bordas)
    ordenar contornos por área, decrescente
    para cada c em contornos:
        se AREA(c) < 0,20 · area_da_foto: pare
        p ← APROXIMAR_POLIGONO(c, tolerância = 2% do perímetro)
        se |p| = 4 e CONVEXO(p):
            retorne ORDENAR_CANTOS(p)
    retorne NULO

função ORDENAR_CANTOS(p):                    // sup-esq, sup-dir, inf-dir, inf-esq
    sup_esq ← ponto com menor x + y
    inf_dir ← ponto com maior x + y
    sup_dir ← ponto com menor y − x
    inf_esq ← ponto com maior y − x
    retorne [sup_esq, sup_dir, inf_dir, inf_esq]
```

3.4 Cálculo da homografia

```
função HOMOGRAFIA(origem[4], destino[4]):
    A ← matriz 8×8; b ← vetor 8
    para i de 0 até 3:
        (x, y)   ← origem[i]
        (x', y') ← destino[i]
        A[2i]   ← [x, y, 1, 0, 0, 0, −x·x', −y·x'];  b[2i]   ← x'
        A[2i+1] ← [0, 0, 0, x, y, 1, −x·y', −y·y'];  b[2i+1] ← y'
    se DET(A) ≈ 0: erro "cantos colineares"
    h ← RESOLVER(A, b)                        // eliminação de Gauss
    retorne [[h1, h2, h3], [h4, h5, h6], [h7, h8, 1]]

função RETANGULO_DESTINO(c):
    W ← média(|c1 − c0|, |c2 − c3|)
    Hd ← média(|c3 − c0|, |c2 − c1|)
    retorne [(0,0), (W−1,0), (W−1,Hd−1), (0,Hd−1)]
```

3.5 Warp inverso com interpolação bilinear

```
função WARP_INVERSO(f, H, W, Hd):
    Hinv ← INVERSA(H)
    saida ← matriz Hd×W
    para v de 0 até Hd−1:
        para u de 0 até W−1:
            [xw, yw, w] ← Hinv · [u, v, 1]ᵀ
            x ← xw / w;  y ← yw / w
            se (x, y) fora de f: saida[v][u] ← 255; continue
            x0 ← ⌊x⌋; y0 ← ⌊y⌋; dx ← x − x0; dy ← y − y0
            saida[v][u] ← (1−dx)(1−dy)·f[y0][x0] + dx(1−dy)·f[y0][x0+1]
                        + (1−dx)dy·f[y0+1][x0] + dx·dy·f[y0+1][x0+1]
    retorne saida
```

3.6 Correção de iluminação e binarização adaptativa

```
função CORRIGIR_ILUMINACAO(f, k):
    se QUADRO_NEGRO(f): f ← 255 − f          // média da imagem < 100
    b ← CONVOLUCAO(f, MEDIA_kxk)
    retorne 255 · min(1, f / (b + ε))

função BINARIZAR_ADAPTATIVO(f, m, C):
    media ← CONVOLUCAO(f, MEDIA_mxm)
    retorne (f < media − C)                  // 1 = escrita
```

Para máscaras grandes (k = 41), a média é calculada com imagem integral (soma acumulada em linhas e colunas), que dá o mesmo resultado da convolução com custo constante por pixel.

3.7 Flood fill iterativo

```
função FLOOD_FILL(B, p0, rotulo, R):          // R: matriz de rótulos (0 = sem rótulo)
    pilha ← [p0]
    area ← 0
    enquanto pilha não vazia:
        p ← DESEMPILHAR(pilha)
        se p fora da imagem ou B[p] = 0 ou R[p] ≠ 0: continue   // caso base
        R[p] ← rotulo; area ← area + 1
        para cada q em N4(p): EMPILHAR(pilha, q)                // 4 chamadas
    retorne area
```

Cada iteração corresponde a uma chamada FF(p, V) da recorrência da Entrega 1 (seção 6.2): o teste do caso base é o mesmo, e empilhar os 4 vizinhos equivale às 4 chamadas recursivas. Por isso valem os Teoremas 1 a 3 (terminação, correção e custo 4n + 1).

3.8 Rotulação e limpeza de manchas

```
função ROTULAR(B):
    R ← matriz de zeros; L ← 0; areas ← []
    para cada pixel p, linha por linha:
        se B[p] = 1 e R[p] = 0:
            L ← L + 1
            areas[L] ← FLOOD_FILL(B, p, L, R)
    retorne (R, L, areas)

função LIMPAR_MANCHAS(B, a_min):
    (R, L, areas) ← ROTULAR(B)
    para cada pixel p com R[p] > 0:
        se areas[R[p]] < a_min: B[p] ← 0
    retorne B
```

3.9 Separação em blocos e módulo de IA

```
função SEPARAR_BLOCOS(B, claro):
    D ← CONVOLUCAO(B, UNS_15x15) > 0          // dilatação
    (R, L, _) ← ROTULAR(D)
    blocos ← []
    para r de 1 até L:
        caixa ← RETANGULO_ENVOLVENTE(pixels com R = r)
        se ÁREA(caixa) ≥ area_min_bloco:
            adicionar RECORTE(claro, caixa) a blocos
    ordenar blocos por (topo, esquerda)
    retorne blocos

função RESUMIR_COM_IA(blocos):
    transcricoes ← []
    para cada bloco em blocos:
        adicionar CHAMAR_MODELO("Transcreva o conteúdo deste trecho de quadro", bloco)
    texto ← JUNTAR(transcricoes)
    retorne CHAMAR_MODELO("A partir desta transcrição de aula, devolva em JSON:
                           tema, resumo em tópicos e explicação didática", texto)
```

4. Especificação das bibliotecas adotadas

O projeto usa seis bibliotecas Python; as funções prontas usadas têm sua matemática documentada na Entrega 1, e os algoritmos avaliados (convolução, homografia, flood fill e rotulação) têm implementação própria.

Biblioteca	Versão mínima	Uso no projeto	Funções principais	Implementado pelo grupo
Python	3.10	Linguagem do protótipo	—	—
NumPy	1.26	Matrizes da imagem e toda a álgebra linear	ndarray, linalg.solve, linalg.inv, cumsum (imagem integral)	Convolução, Sobel, homografia, warp bilinear, iluminação, binarização, flood fill, rotulação
OpenCV (opencv-python)	4.8	Leitura de imagem, contornos e validação dos resultados próprios	imread, cvtColor, findContours, approxPolyDP, getPerspectiveTransform, warpPerspective, adaptiveThreshold, connectedComponents	Usado só para contornos e como referência nos testes
Streamlit	1.30	Interface web: envio da foto, visualização das etapas e do resumo	file_uploader, camera_input, image, columns, download_button	—
Pillow	10.0	Exportação da imagem limpa em PDF	Image.fromarray, Image.save(format="PDF")	—
SDK da API de IA (anthropic)	0.40	Chamada ao modelo multimodal para transcrição, resumo e explicação	messages.create com imagem em base64	Prompts e formato da resposta (JSON)
pytest	8.0	Testes automáticos	—	Testes comparando as funções próprias com o OpenCV

Validação das implementações próprias: cada função feita pelo grupo terá um teste que compara sua saída com a função equivalente do OpenCV na mesma imagem (por exemplo, nossa homografia contra getPerspectiveTransform, com diferença máxima de 10^{-6}). Isso comprova que a matemática da Entrega 1 foi implementada corretamente.

Chave da API de IA: fica em um arquivo .env fora do repositório. Sem chave ou sem internet, o sistema entrega a imagem tratada e avisa que o resumo não está disponível; o processamento visual não depende da IA.

5. Estrutura do repositório e tratamento de erros

O código fica organizado em um pacote por módulo da arquitetura, o que permite dividir o trabalho entre os três integrantes sem conflito.

```
scanner-quadro/
├── app.py                  # interface Streamlit
├── scanner/
│   ├── pipeline.py         # PROCESSAR_FOTO (orquestrador)
│   ├── captura.py          # leitura e redimensionamento
│   ├── preprocessamento.py # cinza, convolução, Gaussiano, Sobel
│   ├── geometria.py        # cantos, homografia, warp inverso
│   ├── realce.py           # iluminação, imagem integral, binarização
│   ├── segmentacao.py      # flood fill, rotulação, limpeza, blocos
│   ├── ia.py               # cliente da API de IA e prompts
│   └── config.py           # parâmetros da seção 2
├── tests/                  # testes contra o OpenCV
├── amostras/               # fotos de quadros para calibração e demo
├── requirements.txt
├── .env.example            # modelo da chave da API
└── README.md               # instalação e execução (Entrega 3)
```

Tratamento de erros e casos limite:

Situação	Como o sistema reage
Quadro não encontrado (nenhum contorno com 4 vértices)	Pede ao usuário que clique nos 4 cantos; se ele não fizer, segue sem retificação
Cantos quase colineares (det A ≈ 0)	Descarta o contorno e tenta o próximo maior
Quadro negro	Detectado pela média de intensidade (< 100); a imagem é invertida antes do realce
Foto muito escura ou desfocada	Aviso na tela pedindo nova foto; o processamento continua
Nenhum bloco de escrita encontrado	Mostra a imagem tratada e informa que não há conteúdo para resumir
API de IA indisponível ou sem chave	Entrega a imagem tratada e o PDF; o resumo aparece como indisponível

Divisão sugerida para a Entrega 3: um integrante em preprocessamento.py e geometria.py, outro em realce.py e segmentacao.py, e o terceiro em ia.py, app.py, testes e README.